SC2002 OBJECT-ORIENTED DESIGN & PROGRAMMING · GROUP ASSIGNMENT

Restaurant Rush

A turn-based restaurant management game for the command line, built in three tagged stages. Design report: UML, design discussion, functional evidence and reflection.

3	22	6
TAGGED, RUNNABLE STAGES	AUTOMATED RULE CHECKS	REPRODUCIBLE SCRIPTED RUNS

RESTAURANT RUSH

LAB GROUP / TEAM
[LAB GROUP] · Group [N]

MEMBERS
[Member 1] · [Member 2] · [Member 3] · [Member 4] · [Member 5]

REPOSITORY
https://github.com/[ORG]/restaurant-rush

BUILD
Java 17+, no external libraries · `./run.sh`

Contents

1	Overview: the game, how to run it, the three stages	3
2	UML class diagram (final design, Stage 3)	4
3	UML sequence diagram: a VIP order during Happy Hour	6
4	Design discussion: abstractions, principles, trade-offs	9
5	How the player's choices change the result	12
6	Development process: stages, tags and change analysis	13
7	Functional evidence: runs, screens and rule checks	15
8	Rule interpretations we documented	23
9	Reflection	24

Before submitting

Fill in the highlighted fields (like this) on the cover, in §6 and in §9, then rebuild with `python3 docs/report/build.py` . Page numbers are filled in by the build.

1 Overview

The player is the restaurant manager. Every turn they decide what each member of staff does. Staff time is scarce, so the real decision is always *who comes first*: seat the new arrival, take an order, or serve the dish that is getting cold?

BASE SETTING (ALWAYS OFFERED)

- **18 turns**, **2 tables**, no early ending
- Waiter, Chef, Cashier: 1 task each per turn
- Arrivals on turns 1, 4, 7, 10, 13, 16
- Salad $9 (1 unit), Burger $15 (1), Pasta $18 (2)
- **Win**: ≥ 4 paid, ≥ $45, avg satisfaction ≥ 60

RUN IT

```
./run.sh              # build and play
./run.sh --script scripts/stage3-victory.txt
./run.sh --plain      # no colour, ASCII
./test.sh             # 22 rule checks
git checkout stage-1  # any stage, then ./run.sh
```

Windows: `run.bat` / `test.bat`. Needs only a JDK (17+); no build tool, no network.

One turn

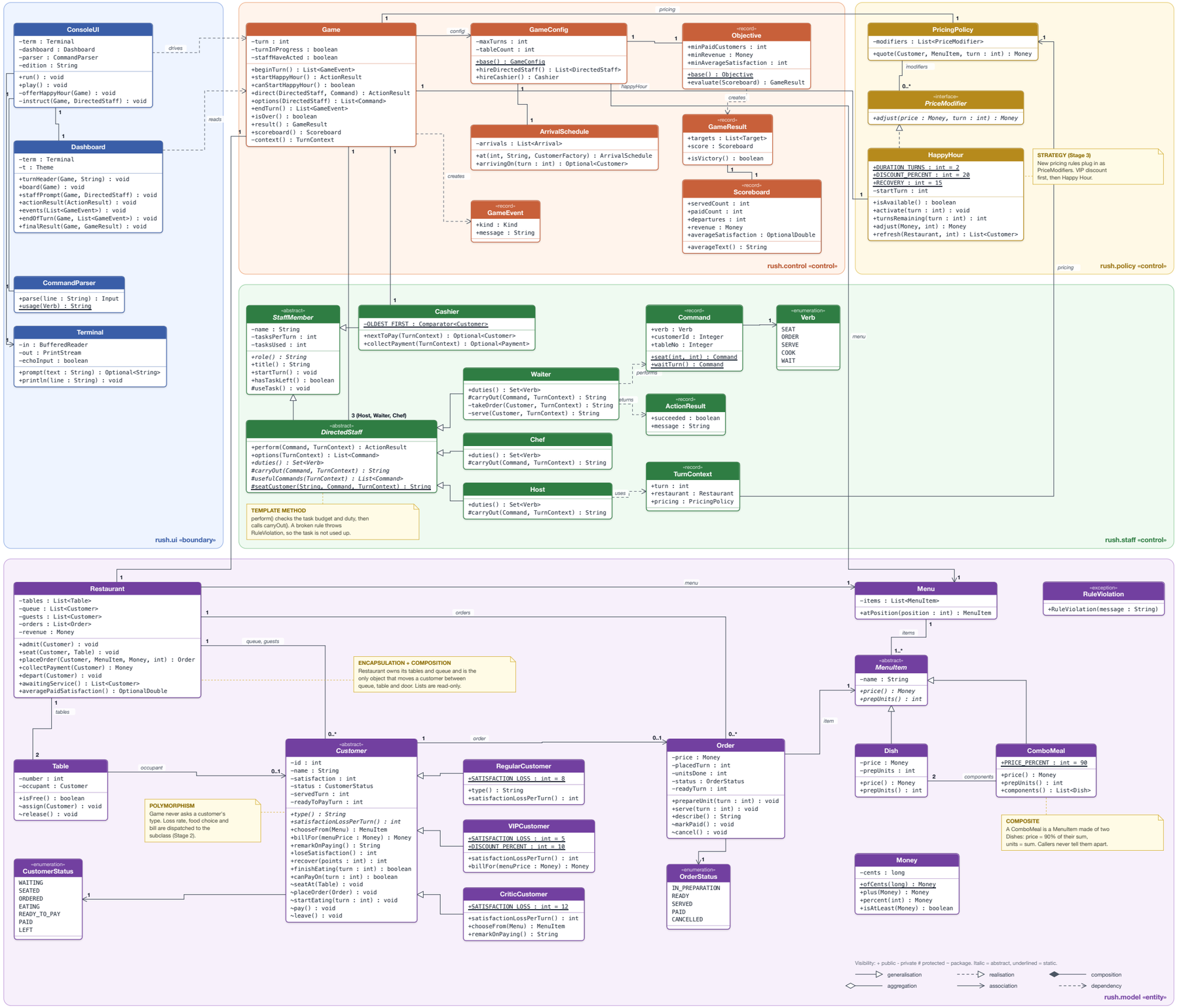
Step	What happens	Where the rule lives
0	Start-of-turn display	`Dashboard.turnHeader/board` (turn, revenue, average, queue, tables, satisfaction bars, order status, staff tasks available, Happy Hour status)
1	Arrivals	`Game.beginTurn` asks `ArrivalSchedule.arrivingOn(turn)` and calls `Restaurant.admit`
2	Manager choices	`ConsoleUI` collects one command per `DirectedStaff`; `DirectedStaff.perform` validates (Host → Waiter → Chef)
3	Kitchen and serving	`Order.prepareUnit` (READY on the last unit) and `Order.serve` (only if READY *before* this turn)
4	Eating and payment	`Customer.finishEating`, then `Cashier.collectPayment` (one per turn, oldest first, only if ready on an earlier turn)
5	Waiting and abandonment	`Customer.loseSatisfaction` (polymorphic rate) and `Restaurant.depart` at 0
6	Result (after turn 18)	`Objective.evaluate(Scoreboard)` → `GameResult`

The three stages

Tag	Adds	Base-setting lineup	Rule checks
`stage-1`	Regular customers, menu, Waiter, Chef, Cashier, full turn sequence, both endings	6 × Regular	12
`stage-2`	`VIPCustomer`, `CriticCustomer`	Regular, **Critic** (T4), **VIP** (T7), 3 × Regular	16
`stage-3`	`Host`, `ComboMeal`, Happy Hour	as Stage 2, plus Host and combos	22

2 UML class diagram: final design (Stage 3)

Every class, attribute, operation and relationship below is taken from the code at tag **`stage-3`**. Getters, `toString()` and private helpers are omitted. Notes annotate encapsulation, polymorphism and composition. Source: `docs/diagrams/class_diagram.py`.



2.1 Reading the class diagram

ENCAPSULATION

`Money` is immutable and stores integer **cents**, so $9.00 + $15.00 is exactly $24.00 and VIP and Happy Hour discounts round once per step.

`Restaurant` is the only object that moves a customer between the queue, a table and the door. `Table.assign/release` and `Customer.seatAt/placeOrder/pay/leave` are package-private (`~`), and every collection is returned read-only. The UI cannot put the game into an impossible state even by accident.

POLYMORPHISM

`Customer` owns the visit life cycle. Subclasses override only three hooks: `satisfactionLossPerTurn()` , `chooseFrom(Menu)` and `billFor(Money)` . `Game` , `Waiter` and `Cashier` never ask a customer's type.

`MenuItem.price()/prepUnits()` are answered differently by `Dish` and `ComboMeal` ; `Order` and `Chef` treat both the same.

COMPOSITION AND AGGREGATION

◆ `Game` owns its restaurant, staff, Happy Hour and pricing policy; they live and die with one game. ◆ `Restaurant` owns its 2 tables and its orders.

◇ `Restaurant` aggregates customers, who arrive from outside. ◇ A `ComboMeal` aggregates 2 existing `Dish` es that also appear on the menu on their own.

INHERITANCE AND INTERFACES

`StaffMember` holds the per-turn task budget. `DirectedStaff` adds the manager-facing template method `perform()` ; `Waiter` , `Chef` and `Host` fill in `duties()` and `carryOut()` . `Cashier` is automatic, so it extends `StaffMember` only.

`PriceModifier` is the one interface, implemented by `HappyHour` .

Layers

`rush.ui` boundary: reads input, draws the screen, no rules · `rush.control` the turn sequence and setting · `rush.staff` what each role may do · `rush.policy` pricing rules · `rush.model` entities and their invariants. Dependencies point one way: `ui → control → staff → policy → model` .

Records (`Command` , `ActionResult` , `TurnContext` , `Objective` , `Scoreboard` , `GameResult` , `GameEvent`) are immutable value carriers; their components are public final by construction.

3 UML sequence diagram

Scenario: the Stage 3 victory run (`scripts/stage3-victory.txt`), following **VIP customer #3 Chloe** from arrival on turn 7 to payment on turn 11, then the result check.

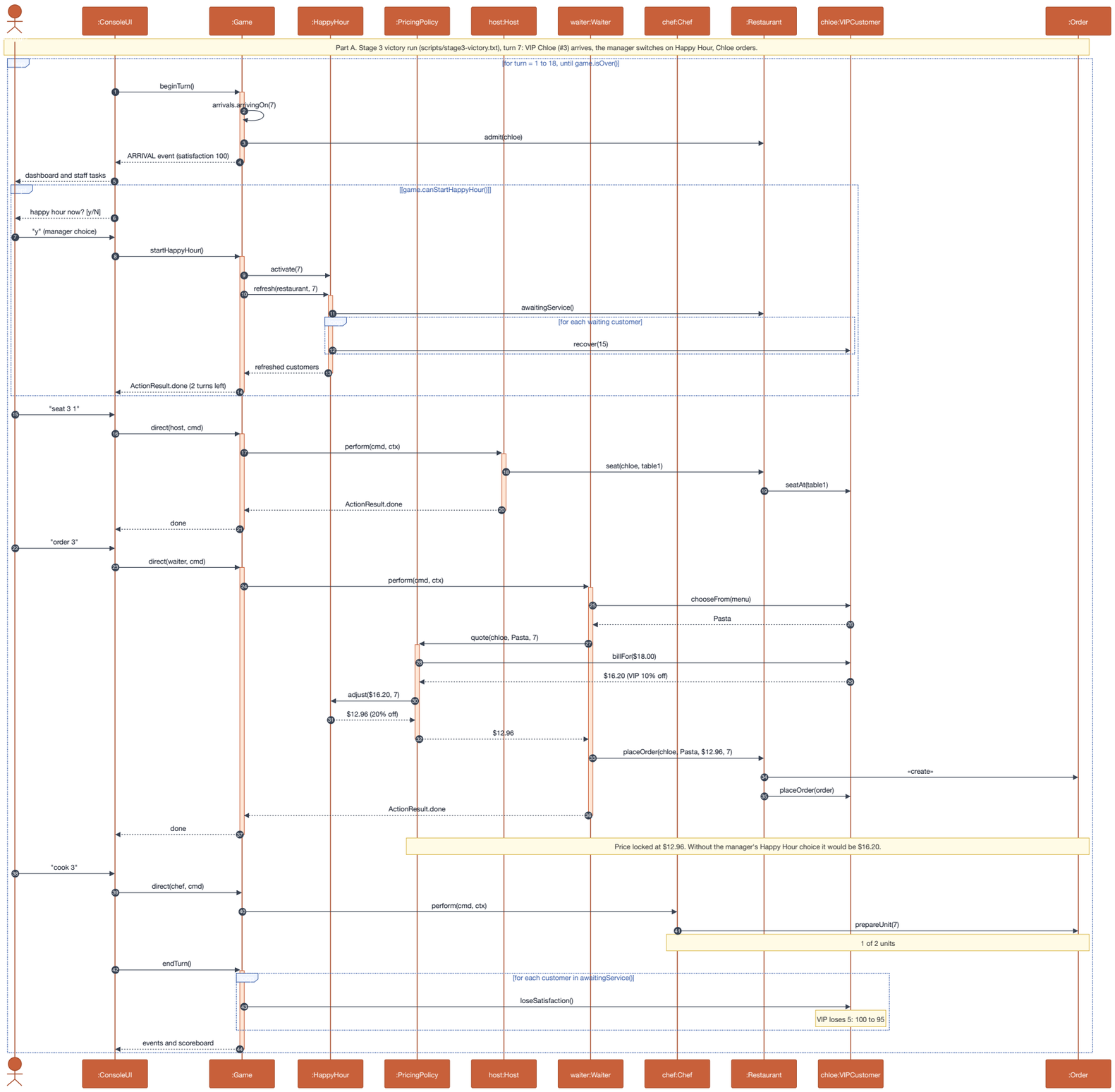
Requirement	Where it appears
Customer arrival and order	Part A, messages 1–4 and 22–36: `arrivingOn` , `admit` , `chooseFrom` , `placeOrder` , `«create» Order`
Waiter and Chef assignments	Part A: `direct(host…)` , `direct(waiter…)` , `direct(chef…)` ; Part B turns 8 and 9
READY then SERVED	Part B: `prepareUnit(8)` sets READY; `serve(9)` guarded by `[status == READY, readyTurn < turn]`
Eating and payment	Part B: `finishEating(10)` ; turn 11 `collectPayment` → `pay()` → `markPaid()` , table released, revenue
Result check	Part B, end: `result()` → `Objective.evaluate(scoreboard())` → `GameResult`
Stage 2/3 behaviour	**VIP pricing** (`billFor` −10%) **and Happy Hour** (`activate` , `refresh/recover` , `adjust` −20%)
Guards and loops	`loop` over turns, waiting customers and recovery; `opt` `[game.canStartHappyHour()]` ; `alt` serve succeeds or is rejected; `opt` Cashier

> **The manager choice that changes the outcome**
>
> On turn 7 the manager answers `y` to Happy Hour *before* the Waiter takes Chloe's order. `PricingPolicy.quote` applies the VIP discount first and then Happy Hour, so the Pasta is locked at **$18.00 × 0.9 × 0.8 = $12.96** instead of $16.20, and the waiting Critic recovers +15 on turns 7 and 8. Over the whole run this choice costs $3.24 of revenue but lifts the final average satisfaction from 75.6 to **82.6** (§5).

The diagram is split in two parts so it stays legible when printed: Part A covers turn 7 (arrival, Happy Hour, seating, ordering, pricing, first unit of cooking, end-of-turn patience). Part B covers turns 8–11 and the final result. Both parts sit inside the same `loop` over turns. Messages use the real method names from the code.

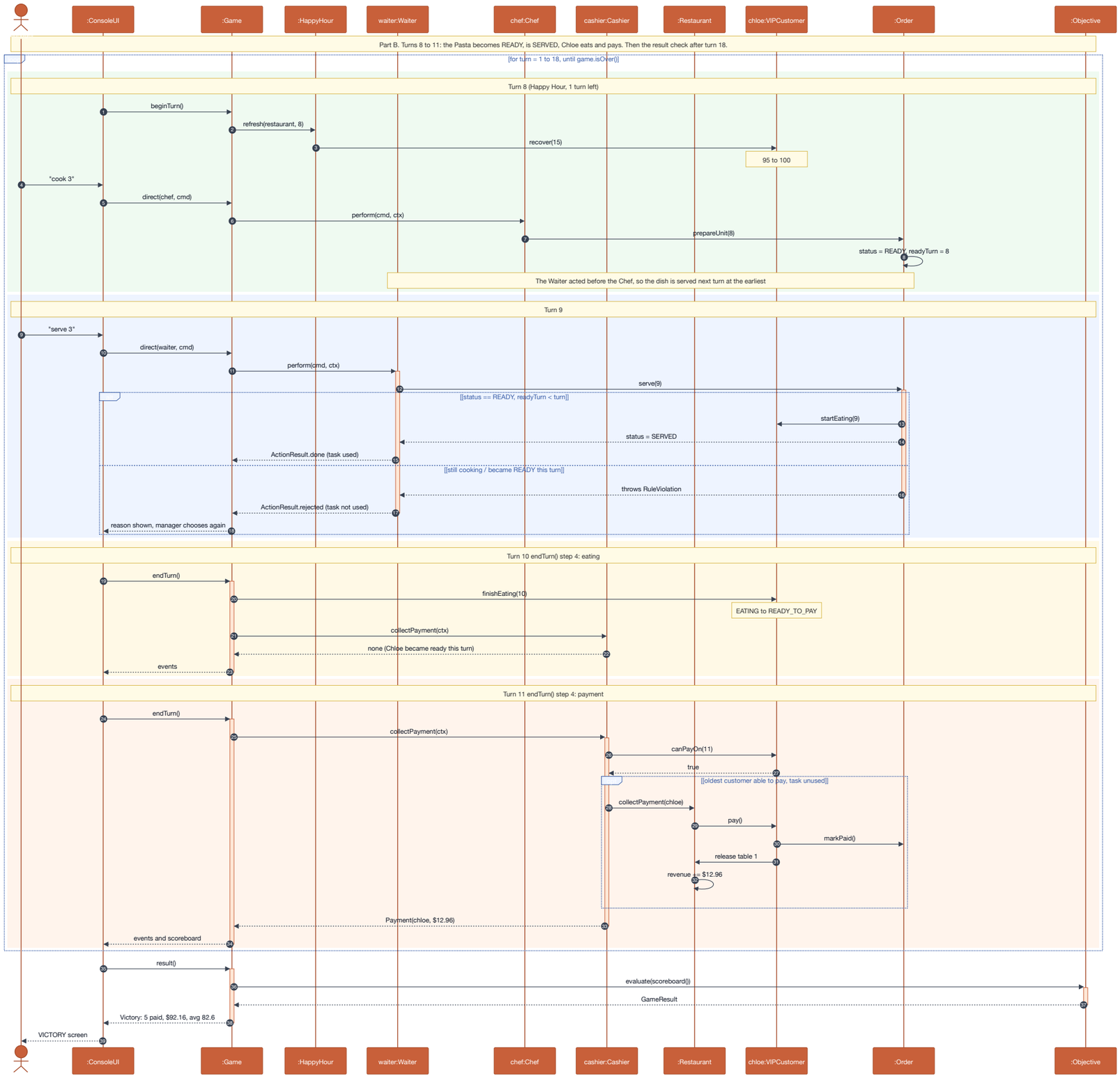
3A · Turn 7: arrival, Happy Hour, seating, ordering, pricing

Source: `docs/diagrams/sequence-1-arrival-order.mmd`



3B · Turns 8–11: READY, SERVED, eating, payment, and the result check

Source: `docs/diagrams/sequence-2-service-payment.mmd`



4 Design discussion

4.1 Key abstractions and their responsibilities

Abstraction	Layer	Responsibility (one reason to change)
`Game`	control	Knows the order of the six steps and nothing else. Every rule it triggers lives in a domain or staff object.
`GameConfig`, `ArrivalSchedule`	control	The base setting: turns, tables, menu, who arrives when (as constructor references), who is hired. Each stage's "registration" happens here.
`Objective`, `Scoreboard`, `GameResult`	control	Targets, the end-of-turn numbers and the Victory/Defeat verdict with a line per target.
`StaffMember` / `DirectedStaff`	staff	Per-turn task budget; the template method that validates and carries out a manager instruction.
`Waiter`, `Chef`, `Host`	staff	The duties of one role and how to carry them out; which commands are currently useful (shown as hints).
`Cashier`	staff	Picks the oldest customer able to pay and takes one payment per turn.
`PricingPolicy`, `PriceModifier`, `HappyHour`	policy	Locks the bill when an order is taken; Happy Hour's once-per-game life cycle, discount and recovery.
`Restaurant`	model	Aggregate root: queue, tables, guests, orders, revenue; keeps them consistent.
`Customer` + 3 subclasses	model	Visit life cycle and satisfaction (base); patience, food choice and bill (subclasses).
`Order`	model	Locked price, preparation progress, the READY → SERVED rule, cancellation.
`MenuItem`, `Dish`, `ComboMeal`, `Menu`	model	What can be ordered; a combo derives its price and units from its dishes.
`Money`, `RuleViolation`	model	Exact cents arithmetic; a player-readable reason why a move breaks a rule.
`ConsoleUI`, `Dashboard`, `CommandParser`, `Terminal`, `Theme`	boundary	Flow of prompts, drawing the state, the shape of commands, I/O and script echo, colours and ASCII fallback.

4.2 Design principles, where the code actually shows them

SRP · SINGLE RESPONSIBILITY

Each rule has exactly one home. "A dish finished this turn cannot be served until next turn" is checked only in `Order.serve`; the Waiter just calls it, and the UI just shows the `RuleViolation` message. `Dashboard` only reads the game and `CommandParser` only checks the *shape* of input (`seat 2 1`), never whether the move is legal. When we changed the end-of-turn layout we touched one class.

OCP · OPEN FOR EXTENSION, CLOSED FOR MODIFICATION

Stage 1 → 2 added VIP and Critic customers with **no change** to `Game`, `Waiter`, `Chef`, `Cashier`, `Restaurant`, `Order` or `Customer`: `git diff stage-1 stage-2 -- src/rush/control/Game.java src/rush/staff` is empty. Stage 3's Host is a new `DirectedStaff` registered in `GameConfig.hireDirectedStaff()`; the console's staff loop prompted it without any change to that loop. Combos needed no change to `Order`, `Chef` or `Waiter`.

LSP · LISKOV SUBSTITUTION

Subclasses cannot break the life cycle: the transitions (`seatAt`, `placeOrder`, `pay`, `leave`) are implemented once in `Customer`, guarded by status checks, and no subclass overrides them. Subclasses only vary the three behaviour hooks, and each override keeps the base contract (a loss rate ≥ 0, an item from the given menu, a bill ≤ the menu price). The test *"all customer types follow the same order and service flow"* runs a VIP through the exact sequence a Regular uses. A `ComboMeal` stands in for any `MenuItem`: the Critic's "most elaborate item" comparator and `Chef`'s unit counting work unchanged.

ISP · INTERFACE SEGREGATION

The Cashier takes no instructions, so it is a `StaffMember` but not a `DirectedStaff` and is not forced to implement `duties()`, `carryOut()` or `options()`. `ConsoleUI` depends only on the narrow `DirectedStaff` API (`perform`, `options`, `hasTaskLeft`). `PriceModifier` has a single method.

DIP · DEPENDENCY INVERSION (PARTLY)

`PricingPolicy` depends on the `PriceModifier` abstraction, not on `HappyHour`. `ArrivalSchedule` depends on a `CustomerFactory` functional interface, so it never names a concrete customer class. The console depends on `DirectedStaff`, not on `Waiter`. **Honest limit:** `Game` still constructs `HappyHour` and `PricingPolicy` itself. Injecting them through `GameConfig` would be the next step if more policies appeared.

4.3 Patterns we used (and why)

Pattern	Where	Why it earns its place
Template Method	`DirectedStaff.perform`	Budget check, duty check, "rejected moves are free", in one place for every role.
Composite	`MenuItem` ← `Dish`, `ComboMeal`	A combo is a menu item made of menu items; price and units are derived, never duplicated.
Strategy (pricing chain)	`PricingPolicy` + `PriceModifier`	Restaurant-wide pricing rules plug in after the customer's own discount, in a fixed order.
Command (light)	`Command` record	Decouples "what the manager typed" from "who validates it"; scripts are just lists of commands.

Factory method references	`ArrivalSchedule.at(4, "Ben", CriticCustomer::new)`	Each stage registers its lineup without the schedule knowing customer classes.

4.4 Trade-offs

Decision	Benefit	Cost we accepted
Domain objects throw `RuleViolation` with a player-facing message	One source of truth per rule; the reason shown to the player is the rule itself; no duplicated "can I?" checks in the UI.	Exceptions on an expected path. Contained: only `DirectedStaff.perform` and `Game.startHappyHour` catch them.
`Customer` abstract from Stage 1, but `MenuItem` concrete until Stage 3	Customer variation was certain and had to be polymorphic. Combos were the *only* reason for a second menu-item kind, so we extracted it when needed (YAGNI).	Stage 3 had to edit `MenuItem` and its construction in `GameConfig` (a 3-line change; callers untouched).
Events are short English strings built in `Game`	Simple; the interface styles them by `GameEvent.Kind` without parsing.	Messages are not localisable. A structured event type per kind would be the alternative.
The dashboard colours customer types by name	Presentation stays out of the domain.	A small switch on `type()` exists in the UI (colour only; an unknown type still displays). There is no type switch anywhere in the rules.
`TurnContext` passed to staff on every call	Staff hold no references to a particular game; easy to test in isolation.	One extra parameter on staff methods.
Staff with nothing useful to do wait automatically (Stage 3)	Fewer meaningless prompts once the Host was added (three roles per turn).	A UI behaviour change between stages, documented in §6.

4.5 An alternative design we considered: the State pattern for customers

Customers move through `WAITING → SEATED → ORDERED → EATING → READY_TO_PAY → PAID` (or `LEFT`). We sketched a State pattern: one class per state (`Waiting`, `Seated`, …), each implementing `seat()`, `order()`, `serve()`, `endOfTurn()` and rejecting what it does not allow.

For

Every transition and its legality is in one place per state; illegal moves fail by design.

Against (why we chose the enum)

The life cycle is strictly linear, and the *type* behaviours (loss rate, bill, food choice) cut across all states, which would mean seven state classes times three customer types to reason about. An enum with guarded transition methods in one abstract class is shorter, easy to test, and keeps the polymorphism where the brief asks for it: in the customer types.

Also rejected: a `CustomerType` enum with `switch` statements in the controller (contradicts "dispatch through the customer objects"), and a Decorator chain for prices (the discount order "VIP first, then Happy Hour" reads more clearly as an explicit list in `PricingPolicy`).

5 How the player's choices change the result

The setting is deterministic, so any difference in outcome comes from the manager's decisions. We replayed the real `Game` on the base setting with every fixed priority for the Waiter, the Chef cooking an order on the turn it is taken or not, and, in Stage 3, using the Host or not and different Happy Hour turns.

Stage 1

Waiter priority	Chef cooks same turn	Paid	Revenue	Avg sat.	Result
serve › order › seat	yes	5	$66.00	79.2	**Victory**
serve › order › seat	no	4	$51.00	66.0	**Victory**
seat › order › serve	no	4	$51.00	60.0	**Victory (just)**
seat › serve › order	no	3	$42.00	60.0	**Defeat**

Stage 2 (Critic and VIP)

Waiter priority	Chef cooks same turn	Paid	Revenue	Avg sat.	Result
serve › order › seat	yes	5	$67.20	76.0	**Victory**
seat › order › serve	no	4	$52.20	62.5	**Victory**
seat › serve › order	no	3	$43.20	59.3	**Defeat on all three**

Stage 3 (Host, combos, Happy Hour)

Waiter priority	Chef same turn	Host	Happy Hour	Paid	Revenue	Avg	Result
order › serve › seat	yes	yes	turn 7	5	$92.16	82.6	**Victory**
order › serve › seat	yes	yes	never	5	$95.40	75.6	**Victory**
serve › order › seat	yes	yes	turn 7	4	$67.86	84.3	**Victory**
seat › serve › order	no	**yes**	turn 7	4	$67.86	81.0	**Victory**
seat › serve › order	no	**no**	turn 7	3	$49.50	68.7	**Defeat**
seat › serve › order	no	no	turn 14	3	$49.50	57.0	**Defeat**

Serve before you seat

A served customer stops losing satisfaction and frees a table two turns later. Seating first only moves the waiting from the queue to a table.

The Host changes the best priority

With a Host doing the seating, *take the order first* beats *serve first* (5 paid vs 4). Using the Host alone turns the rookie strategy from Defeat into Victory.

Happy Hour is a trade

Switched on at turn 7 it costs $3.24 of revenue but lifts the average from 75.6 to 82.6. Switched on at turn 14 it reaches nobody who can still pay.

6 Development process

Each stage is a tagged, runnable commit. Every tag compiles on its own, passes its own rule checks and replays its own victory and defeat scripts (verified from clean `git archive` checkouts of each tag).

Commit history

Commit	Date	Message	Tag
`7727982`	2026-10-08	Stage 1: domain model (Money, menu, customers, orders, tables, restaurant)	
`a724465`	2026-10-08	Stage 1: staff roles and the six-step turn controller	
`3a27533`	2026-10-08	Stage 1: command-line interface with dashboard and command parser	
`ebdc913`	2026-10-08	Stage 1: rule checks, reproducible victory/defeat scripts and test notes	`stage-1`
`a721c2d`	2026-10-08	Stage 2: VIP and Critic customers dispatched through Customer subclasses	`stage-2`
`13d6dc5`	2026-10-08	Stage 3: Host, ComboMeal (MenuItem becomes abstract), Happy Hour pricing policy	`stage-3`

Compare links: https://github.com/[ORG]/restaurant-rush/compare/stage-1...stage-2 and …/compare/stage-2...stage-3. Locally: `git diff stage-1 stage-2 --stat`.

Stage 1 → Stage 2: customer behaviour

File	Change	Reason
`model/VIPCustomer.java`	new	Loss 5, `billFor` 10% off, own remark.
`model/CriticCustomer.java`	new	Loss 12; overrides `chooseFrom` (most elaborate dish) and gives a star review.
`model/Money.java`	+5 lines	`percent(int)`, rounded half-up, needed for the first discount.
`control/GameConfig.java`	2 lines	**Registration**: Ben (T4) becomes a Critic, Chloe (T7) a VIP. Still six arrivals.
`ui/Dashboard.java`	small	Colour per customer type, and a guest-type legend in the help screen (presentation only).
`Main.java`, tests, scripts, docs	updated	Edition label; 4 new rule checks; Stage 2 scripts and notes.
`Game`, `Customer`, all staff, `Restaurant`, `Order`	**unchanged**	Behaviour is dispatched through the new subclasses (OCP).

Stage 2 → Stage 3: new role and menu policy

File	Change	Reason
`staff/Host.java`	new	A `DirectedStaff` whose only duty is `SEAT`; reuses the shared seating helper.

`model/Dish.java`, `model/ComboMeal.java`	new	The concrete dish moved into `Dish`; `ComboMeal` derives 90% price and summed units from two dishes.
`model/MenuItem.java`	**rewritten**	Concrete → abstract (`price()`, `prepUnits()`). Same public API, so `Order`, `Chef`, `Waiter` and the Critic's comparator did not change.
`policy/PriceModifier.java`, `policy/HappyHour.java`	new	Once per game, two turns, 20% off orders taken, +15 recovery at turn start for those still waiting.
`policy/PricingPolicy.java`	changed	Now applies a list of `PriceModifier`s after `billFor` (VIP first, then Happy Hour).
`model/Customer.java`	+14 lines	`recover(points)`, capped at 100, only for customers still awaiting service.
`control/Game.java`	changed	Owns `HappyHour`; `startHappyHour()` only before any staff acts; recovery in `beginTurn()`.
`control/GameConfig.java`	small	**Registration**: menu built from `Dish`es plus two combos; Host hired first.
`control/GameEvent.java`	1 line	`POLICY` event kind for Happy Hour recovery.
`ui/ConsoleUI.java`, `ui/Dashboard.java`	changed	Happy Hour prompt and status line (remaining duration); auto-wait for staff with nothing useful to do.
`Waiter`, `Chef`, `Cashier`, `DirectedStaff`, `Restaurant`, `Order`	**unchanged**	Combos and the Host fit the existing abstractions.

Earlier behaviour still works

Stage 1's worked example (Pasta from arrival to payment, satisfaction 92 → 84 → 76, paid $18.00 on turn 6), abandonment, cashier order and "READY then SERVED next turn" are still checked by the Stage 3 test suite on custom settings, and all pass.

7 Functional evidence

Every screen below is real output, captured by replaying the scripts at their own tag (`docs/report/build.py`). Full plain-text transcripts are in `docs/transcripts/`.

7.1 Launch, dashboard and robust input (Stage 1, turn 1)

The start-of-turn display shows the turn, revenue against target, paid count, average satisfaction (`N/A` before anyone pays), the queue, both tables, each customer's satisfaction bar, the kitchen, and the tasks each member of staff can do right now. Two bad inputs are explained and do not use up the Waiter's task.

Stage 1: launch, first dashboard and two rejected inputs (turn 1)

```
RESTAURANT RUSH
a turn-based restaurant management game · Stage 1 · Basic operation

1  Play the base setting
2  How to play
3  Quit

› 1

TURN 1 of 18  ▰▱▱▱
```

7.2 READY → SERVED, and a constrained decision (Stage 1, turns 9–10)

Turn 9: `serve 3` is rejected because the Pasta is 1/2 cooked; the Chef finishes it, but the Waiter has already acted. Turn 10: Dev arrives just as the Pasta is READY. With one Waiter, the manager serves Chloe and leaves Dev in the queue for a turn.

Stage 1, turn 9: serve rejected (still cooking), then the Chef makes it READY

```
◆ WAITER WENDY  seat, order, serve, wait
  try wait
waiter › serve 3
  ✘ #3 Chloe's Pasta is still cooking (1/2 units done).
    Nothing was used up. Choose again.
waiter › wait
  ✔ Waiter Wendy waits this turn.

◆ CHEF CARLO  cook, wait
  try cook 3  ·  wait
chef › cook 3
  ✔ Carlo finishes #3 Chloe's Pasta (2/2). It is READY.

─── End of turn 9 ──────────────────────────────────────────
  ▾ #3 Chloe waiting for pasta: satisfaction 84 -> 76 (−8)

Served 2 · Unhappy departures 0 · Revenue $24.00 · Avg sat. 84.0 · Turns 9/18
```

Stage 1, turn 10: Dev arrives as the Pasta is READY; one Waiter, two needs

```
┌──────────────────────────────────────────────────────────┐
│ TURN 10 of 18  ▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱   Stage 1 · Basic operation │
├──────────────────────────────────────────────────────────┤
│ Revenue $24.00/$45.00   Paid 2/4   Avg satisfaction 84.0/60   Left 0 │
└──────────────────────────────────────────────────────────┘

  ▸ #4 Dev (Regular) arrives and joins the queue with satisfaction 100.

WAITING QUEUE ─────────────────────────────────────────────
   #4 Dev    Regular  ▰▰▰▰▰▰▰▰▰▰ 100  Waiting for a table since turn 10

TABLES ────────────────────────────────────────────────────
 T1  #3 Chloe  Regular  ▰▰▰▰▰▰▰▰▰▰
```

7.3 Payment, victory and defeat (Stage 1)

Chloe was served on turn 10, finished eating on turn 11 and pays on turn 12, freeing table 1.

Stage 1: end of turn 12, the Cashier collects Chloe's Pasta

```
── End of turn 12 ──────────────────────────────────────────────────
  $ Cassie collects $18.00 from #3 Chloe (satisfaction 76 recorded). Table 1
    is free. "Not bad at all."
  ▾ #4 Dev waiting for salad: satisfaction 84 -> 76 (-8)

Served 3 · Unhappy departures 0 · Revenue $42.00 · Avg sat. 81.3 · Turns 12/18
```

Stage 1 victory run: result screen

```
 ▗▖  ▗▖ ▗▄▄▖▗▄▄▄▖ ▗▄▖ ▗▄▄▖▗▖ ▗▖
 ▐▌  ▐▌▐▌   ▐▌   ▐▌ ▐▌▐▌   ▐▌▗▞▘
 ▐▌  ▐▌ ▝▀▚▖▐▛▀▀▘▐▌ ▐▌▐▌   ▐▛▚▖
  ▝▚▞▘ ▗▄▄▞▘▐▙▄▄▖▝▚▄▞▘▝▚▄▄▖▐▌ ▐▌

The dining room hums. Every target was met.

TARGETS ──────────────────────────────────────────────────────────
  ✔ Paid customers                5   at least 4
  ✔ Revenue                  $66.00   at least $45.00
  ✔ Average satisfaction       79.2   at least 60

GUESTS ───────────────────────────────────────────────────────────
  Guest       Type      Ordered              Bill  Sat.  Outcome
  #1 Ada      Regular   Salad               $9.00    84  Paid
  #2 Ben      Regular   Burger             $15.00    84  Paid
  #3 Chloe    Regular   Pasta              $18.00    76  Paid
  #4 Dev      Regular   Salad               $9.00    76  Paid
  #5 Elif     Regular   Burger             $15.00    76  Paid
  #6 Farid    Regular   Pasta              $18.00    76  Waiting for Pasta

Served 5 · Unhappy departures 0 · Revenue $66.00 · Avg sat. 79.2 · Turns 18/18
```

Stage 1 defeat run: result screen

```
 ▗▄▄▄ ▗▄▄▄▖▗▄▄▄▖ ▗▄▖▗▄▄▄▖
 ▐▌  ▐▌   ▐▌   ▐▌ ▐▌▐▌  ▐▌
 ▐▌  ▐▛▀▀▘▐▛▀▀▘▐▛▀▜▌▐▌  ▐▌
 ▐▙▄▄▀▐▙▄▄▖▐▌   ▐▌ ▐▌▐▙▄▄▀

The restaurant fell short of its targets this time.

TARGETS ──────────────────────────────────────────────────────────
  ✘ Paid customers                3   at least 4
  ✘ Revenue                  $42.00   at least $45.00
  ✔ Average satisfaction       60.0   at least 60

GUESTS ───────────────────────────────────────────────────────────
  Guest       Type      Ordered              Bill  Sat.  Outcome
  #1 Ada      Regular   Salad               $9.00    68  Paid
  #2 Ben      Regular   Burger             $15.00    60  Paid
  #3 Chloe    Regular   Pasta              $18.00    52  Paid
  #4 Dev      Regular   Salad               $9.00    44  Ready to pay
  #5 Elif     Regular   Burger             $15.00    52  Waiting for Burger
  #6 Farid    Regular   –                      –    76  Waiting for a table

Served 4 · Unhappy departures 0 · Revenue $42.00 · Avg sat. 60.0 · Turns 18/18
```

Victory script: serve › order › seat, the Chef cooks immediately → 5 paid, $66.00, average 79.2. Defeat script: seat › serve › order, the Chef never cooks on the turn an order is taken → 3 paid, $42.00.

7.4 VIP and Critic behaviour (Stage 2)

The Critic (#2 Ben) ordered Pasta although the rotation would give guest #2 a Burger, and lost 12 per turn while it cooked. On turn 7 the VIP arrives. The manager serves the Critic, because the VIP loses only 5 per turn. On turn 9 the Critic pays and posts a review; on turn 13 the VIP pays **$16.20**.

Stage 2, turn 7: the Critic's Pasta is READY just as the VIP arrives

```
TURN 7 of 18  ▰▰▰▰▰▰▰▱▱▱
```

7.5 Stage 3 features: Host, combos and Happy Hour

Turn 7: Happy Hour is switched on before anyone acts, the Host seats the VIP, the Waiter takes the order in the same turn ($18.00 × 0.9 × 0.8 = **$12.96**), and the Critic, waiting for the 3-unit **Salad + Pasta Combo ($24.30)**, recovers +15. Turn 8: the header shows **1 turn left** and both waiting guests recover again.

Stage 3, turn 7: Happy Hour, the Host seats, the VIP order is $12.96

```
TURN 7 of 18  ▰▰▰▰▰▰▰▱▱▱
```

7.6 Victory and defeat (Stage 3)

Stage 3 victory run: result screen

```
VICTORY
The dining room hums. Every target was met.

TARGETS
  ✔ Paid customers                5   at least 4
  ✔ Revenue                  $92.16   at least $45.00
  ✔ Average satisfaction       82.6   at least 60

GUESTS
  Guest      Type     Ordered               Bill  Sat.   Outcome
  #1 Ada     Regular  Salad                $9.00    92   Paid
  #2 Ben     Critic   Salad + Pasta Combo $24.30    82   Paid
  #3 Chloe   VIP      Pasta               $12.96    95   Paid
  #4 Dev     Regular  Salad + Burger Combo $21.60    68   Paid
  #5 Elif    Regular  Salad + Pasta Combo $24.30    76   Paid
  #6 Farid   Regular  Salad                $9.00    84   Eating

Served 6 · Unhappy departures 0 · Revenue $92.16 · Avg sat. 82.6 · Turns 18/18
```

Stage 3 defeat run: result screen

```
DEFEAT
The restaurant fell short of its targets this time.

TARGETS
  ✘ Paid customers                3   at least 4
  ✔ Revenue                  $49.50   at least $45.00
  ✘ Average satisfaction       57.0   at least 60

GUESTS
  Guest      Type     Ordered               Bill  Sat.   Outcome
  #1 Ada     Regular  Salad                $9.00    68   Paid
  #2 Ben     Critic   Salad + Pasta Combo $24.30    28   Paid
  #3 Chloe   VIP      Pasta               $16.20    75   Paid
  #4 Dev     Regular  Salad + Burger Combo $17.28    74   Ready to pay
  #5 Elif    Regular  Salad + Pasta Combo $24.30    68   Waiting for food
  #6 Farid   Regular  –                       –     76   Waiting for a table

Served 4 · Unhappy departures 0 · Revenue $49.50 · Avg sat. 57.0 · Turns 18/18
```

The defeat script ignores the Host and switches Happy Hour on at turn 14, too late to help anyone who can still pay: 3 paid, average 57.0.

7.7 Automated rule checks, run at each tag

`./test.sh` compiles `src` and `test` with plain `javac` (no JUnit download needed) and checks the rules against the real objects, including the brief's worked example and the pricing examples. The scripted runs are replayed through the real console.

STAGE-1

PASS money is kept in cents and shown with two decimals

PASS worked example 3.3: Pasta from arrival to payment

PASS a dish finished this turn is served next turn at the earliest

PASS an invalid choice explains why and does not use the task

PASS the Waiter completes at most one task per turn

PASS the Cashier takes one payment per turn, oldest first

PASS a customer at 0 leaves, order cancelled, table freed, no revenue

PASS served, eating and paying customers lose no satisfaction

PASS customers choose their own food by rotating through the menu

PASS the game always runs all 18 turns

PASS scripted victory run ends in Victory

PASS scripted defeat run ends in Defeat

12 passed, 0 failed

STAGE-2

PASS money is kept in cents and shown with two decimals

PASS worked example 3.3: Pasta from arrival to payment

PASS a dish finished this turn is served next turn at the earliest

PASS an invalid choice explains why and does not use the task

PASS the Waiter completes at most one task per turn

PASS the Cashier takes one payment per turn, oldest first

PASS a customer at 0 leaves, order cancelled, table freed, no revenue

PASS served, eating and paying customers lose no satisfaction

PASS customers choose their own food by rotating through the menu

PASS the game always runs all 18 turns

PASS VIP: loses 5 per turn and gets 10% off

PASS Critic: loses 12 per turn and orders the most elaborate dish

PASS base setting has six arrivals including a VIP and a Critic

PASS all customer types follow the same order and service flow

PASS scripted victory run ends in Victory

PASS scripted defeat run ends in Defeat

16 passed, 0 failed

STAGE-3

PASS money is kept in cents and shown with two decimals

PASS worked example 3.3: Pasta from arrival to payment

PASS a dish finished this turn is served next turn at the earliest

PASS an invalid choice explains why and does not use the task

PASS the Waiter completes at most one task per turn

PASS the Cashier takes one payment per turn, oldest first

PASS a customer at 0 leaves, order cancelled, table freed, no revenue

PASS served, eating and paying customers lose no satisfaction

PASS customers choose their own food by rotating through the menu

PASS the game always runs all 18 turns

PASS VIP: loses 5 per turn and gets 10% off

PASS Critic: loses 12 per turn and orders the most elaborate dish

PASS base setting has six arrivals including a VIP and a Critic

PASS all customer types follow the same order and service flow

PASS combo: summed units, 90% of the combined price

PASS Host seats one customer in addition to the Waiter

PASS Happy Hour: once per game, this turn and the next

PASS Happy Hour: 20% off after the VIP discount, locked when ordered

PASS Happy Hour: waiting customers recover 15, capped at 100

PASS Happy Hour only starts before any staff member acts

PASS scripted victory run ends in Victory

PASS scripted defeat run ends in Defeat

22 passed, 0 failed

8 Rule interpretations we documented

Topic	What the game does
Food choice (all stages)	Deterministic rotation: guest #n orders item n on the menu board, wrapping around. Shown when the order is taken.
Critic's special behaviour	Order preference: always the item with the most preparation units, the dearer one on a tie (Pasta in Stage 2, Salad + Pasta Combo in Stage 3). Cosmetic 1–5 star review on payment (no effect on scoring).
Combo choice rule	Combos follow the dishes on the board, so the rotation gives guests #4 and #5 the two combos.
Combos on the menu	Salad + Burger ($21.60, 2 units), Salad + Pasta ($24.30, 3 units).
Display order	The scheduled arrival (step 1) is processed, then the start-of-turn dashboard is drawn, so the manager sees the new customer before choosing.
Host and Waiter order	Host, then Waiter, then Chef. The Waiter may take the order of a customer the Host seated this turn (consistent with the Chef working on an order the Waiter took this turn).
Happy Hour start	Offered at the start of each turn until used, before any staff member acts. Recovery is applied immediately on the first turn and at the start of the second.
"Waiting for a table or food"	`WAITING`, `SEATED` (not yet ordered) and `ORDERED` (including a READY but unserved dish). The same set loses satisfaction at turn end.
Rounding	Half-up to the nearest cent after each percentage step: VIP Burger in Happy Hour $15.00 → $13.50 → $10.80.
Seating	`seat 2` uses the first free table; `seat 2 1` names one.
Auto-wait (Stage 3)	A staff member whose only possible action is `wait` waits without a prompt.
Satisfaction display	A paid customer's satisfaction is frozen when they pay, so the average equals the values recorded at payment.

9 Reflection

What worked

- **Designing from the published brief.** Because all three stages were known up front, Stage 1 already separated the customer *life cycle* from customer *behaviour*. Stage 2 then touched only configuration, one value-object helper and presentation.
- **Rules as exceptions with reasons.** Putting each rule in the object that owns the data (`Order.serve`, `Table.assign`, `HappyHour.activate`) gave us clear error messages for free and kept the UI free of rules.
- **Scripts as tests.** Recording runs as plain command files made every victory and defeat reproducible for the TA, the video and the automated checks alike.

What we would do differently

- Inject `PricingPolicy` and its modifiers through `GameConfig` instead of building them in `Game`, so a new policy needs only registration.
- Use structured event objects instead of English strings, which would let tests assert on events rather than text.
- Write the strategy-replay tool (§5) as part of the test suite earlier: it found a balance issue (the Host changes the best Waiter priority) that we only noticed late.

Individual contributions

Member	Main contributions	Video segment
[Member 1]	[e.g. domain model, Money, Order rules]	[1, 3]
[Member 2]	[e.g. staff roles, turn controller]	[2]
[Member 3]	[e.g. VIP and Critic, tests]	[4]
[Member 4]	[e.g. Host, combos, Happy Hour]	[5, 6]
[Member 5]	[e.g. UML, report, README]	[7]

Personal reflections

[Each member: two or three sentences on what you learned about OO design, for example how the Stage 2 change confirmed or challenged your Stage 1 design.]